

Installation de BetterAuth dans un projet NextJS avec Drizzle

⚠ Attention, il existe toujours différentes façon de configurer une bibliothèque. Le but de ce guide est surtout de vous montrer une façon simple afin de vous faire gagner du temps, mais il faut que vous vous l'appropriiez : vous pouvez changer le nom des fichiers et/ou l'organisation de vos dossiers par exemple.

1. Installer la lib

```
pnpm i better-auth
```

2. Ajouter les variables d'environnement comme précisé dans [leur guide d'installation](#)

💡 Pour notre projet, on va gérer l'authentification côté serveur.

3. Créer un fichier `auth.ts` qui initialise better-auth comme ci-dessous :

```
import {betterAuth} from "better-auth";
import {drizzleAdapter} from "better-auth/adapters/drizzle";
import {db} from "@/lib/db/drizzle"; // Change l'import en fonction de TON projet
import * as schema from "@/lib/db/schema"; // Change l'import en fonction de TON projet
import {nextCookies} from "better-auth/next-js";

export const auth = betterAuth({
  emailAndPassword: {
    enabled: true, // On active les comptes par email et mot de passe
  },
  database: drizzleAdapter(db, {
    provider: "pg",
    schema, // Ajoute ton schéma de DB
  }),
  plugins: [nextCookies()], // ⚠ Permet de sauvegarder les cookies
});
```

4. Créer le schéma de base de données avec la commande

```
pnpx @better-auth/cli generate
```

⚠ Attention, la commande génère un fichier `./auth-schema.ts`, copie bien son contenu avec le reste de tes schémas drizzle.

5. Créer les tables en db avec drizzle

```
pnpm db:push
```

6. Créer des composants clients ("use client") pour le formulaire de signin et signup

7. Créer des serveurs actions qui utilisent les données du formulaire pour signin, signup et signout les users, comme ci-dessous :

```
"use server";
import {auth} from "@/lib/auth";
import {headers} from "next/headers";
import {redirect} from "next/navigation";

export const signup = async (formData: FormData) => {
  const name = formData.get("name") as string;
  const email = formData.get("email") as string;
  const password = formData.get("password") as string;
  if (!name && !email && !password) {
    throw Error("Name, email and password are required");
  }
  const response = await auth.api.signUpEmail({
    body: {
      name,
      email,
      password,
    },
    asResponse: true,
  });

  if (!response.ok) {
    console.error("Sign in failed:", await response.json());
    redirect("/auth/signup?error=true");
  }

  redirect("/"); // on redirige vers la home page une fois connecté
};

export const signin = async (formData: FormData) => {
  const email = formData.get("email") as string;
  const password = formData.get("password") as string;
  if (!email && !password) {
    throw Error("email and password are required");
  }
  const response = await auth.api.signInEmail({
    body: {
      email,
      password,
    },
    asResponse: true,
  });
}
```

```
});

if (!response.ok) {
  console.error("Sign in failed:", await response.json());
  redirect("/auth/signin?error=true");
}

redirect("/"); // on redirige vers la home page une fois connecté
};

export const signout = async () => {
  await auth.api.signOut({headers: await headers()}); // attention à
  bien passer les headers !
};
```

8. Maintenant, tu peux utiliser `auth.api.getSession()` pour récupérer les information de l'utilisateur •ice connecté•e. Si la fonction retourne `null`, c'est que l'utilisateur •ce n'est pas connecté•e

```
import {auth} from "@/lib/auth";
import {headers} from "next/headers";

export default async function Home() {
  const session = await auth.api.getSession({headers: await headers()});

  return (
    <div>
      <pre>{session ? JSON.stringify(session.user, null, 2) : "Not
connected"}</pre>
    </div>
  );
}
```